Bagging vs Boosting

Contents

1. Overview 2
2. Difference 1: the type of base model 3
   - 2.1 Boosting: high bias, low variance models 3
   - 2.2 The rule of thumb 3
   - 2.3 Watching the two errors move 4
3. Difference 2: parallel against sequential learning 5
4. Difference 3: the weight of each base model 7
5. Summary 8
6. Sources 8
7. Key terms 9

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Boosting.
- **Compare with:** Bagging; Stacking and blending.

1. Overview

Key point: Bagging (G-251) and boosting differ on three points: the kind of base model (low bias and high variance, against high bias and low variance; bias and variance are the two kinds of error a model can make), the way they learn (in parallel, against in sequence), and the weight of each base model in the vote (equal, against earned).

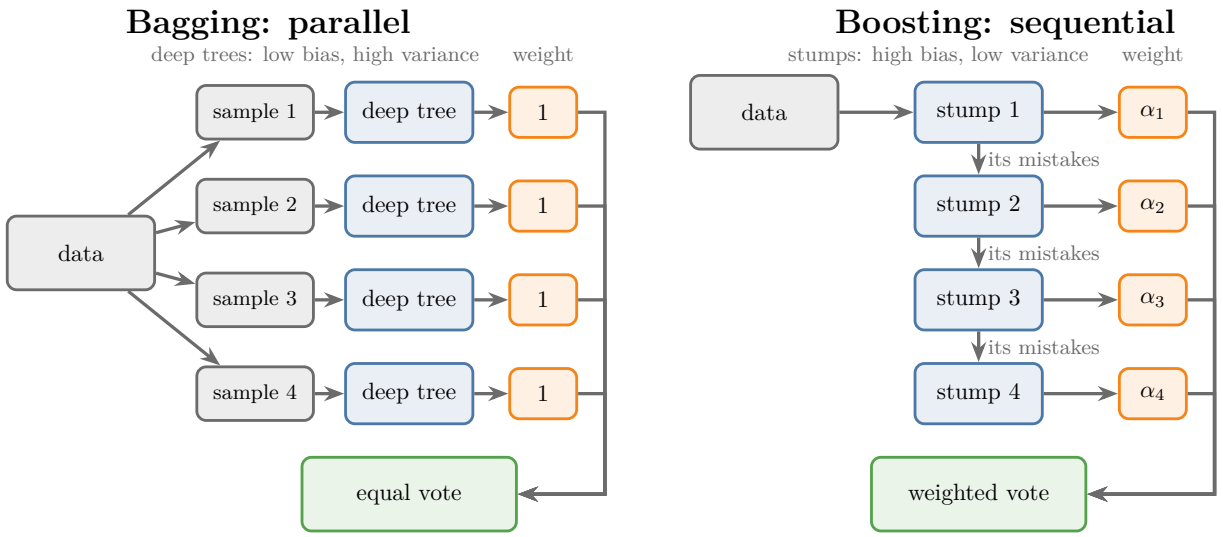


Figure 1: Bagging trains deep trees side by side on random samples and gives each an equal vote; boosting trains stumps one after another, each on the previous one's mistakes, and weights each vote by its alpha (a number saying how much that model's vote counts)

On tables of data, tree ensembles (many decision trees whose answers are combined) built by bagging (**random forests** (G-1611)) and by boosting (**gradient boosting** (G-859), **XGBoost** (G-2133)) are among the strongest models: a benchmark on 45 medium-sized datasets found them ahead of deep networks (Grinsztajn et al. 2022). "What is the difference between bagging and boosting?" is also a common interview question.

We have now seen both: bagging and random forests, and boosting through **AdaBoost** (G-167). This Note puts them side by side on three points (Figure 1). The Notebook (`ML-113-bagging-vs-boosting.ipynb`) runs a small experiment on the first point.

2. Difference 1: the type of base model

> **Key point:** Bagging starts from low-bias, high-variance models and lowers their variance; boosting starts from high-bias, low-variance models and lowers their bias.

We want low bias and low variance, but a single model usually trades one for the other (the bias-variance trade-off). **Bagging** (G-251) takes **low-bias, high-variance** models (G-1132), such as fully grown trees or KNN (which predicts from the k closest points) with a small k, and averages away much of their variance (how bagging lowers variance). Boosting attacks the trade-off from the opposite end.

2.1 Boosting: high bias, low variance models

> **Key point:** Boosting uses models that are too simple to fit the data well but are stable, such as shallow trees and decision stumps.

Boosting needs base models with **high bias and low variance**:

- a **shallow decision tree** (G-1785), whose depth is very small;
- above all the **decision stump** (G-559), a tree of depth 1 (**decision stumps** (G-559)).

A stump is not good on the training data, but it hardly changes when the data changes. Adding many stumps in sequence, each fixing the last one's mistakes, lowers the bias step by step while the variance stays fairly low: on noisy data it creeps up only with very many stumps (average test accuracy 0.846 with 50 stumps, 0.837 with 1,500, in the experiment on the number of stages). ESL §15.2 states the same contrast: bagged trees improve only through lower variance, while boosting grows its trees adaptively to remove bias.

2.2 The rule of thumb

> **Key point:** A model that is very good on the training data but unstable calls for **bagging** (G-251); a model that is stable but weak calls for boosting.

If the algorithm is...	it is...	so use
very good on the training data, unstable when the data changes	low bias, high variance	bagging
not so good on the training data, stable when the data changes	high bias, low variance	boosting

The type of base model is the most important of the three differences.

Extra: Swapping the base models shows why. On the noisy circles data of the AdaBoost experiment (two classes arranged as an inner disc and an outer ring, with some labels flipped at random), with 100 base models each and 10-fold cross-validation (G-510; the data is split into 10 parts, and each part in turn is the test set while the other nine train):

Base model	Alone	Bagging	AdaBoost
stump	0.566	0.738	**0.814**
fully grown tree	0.746	**0.808**	0.750

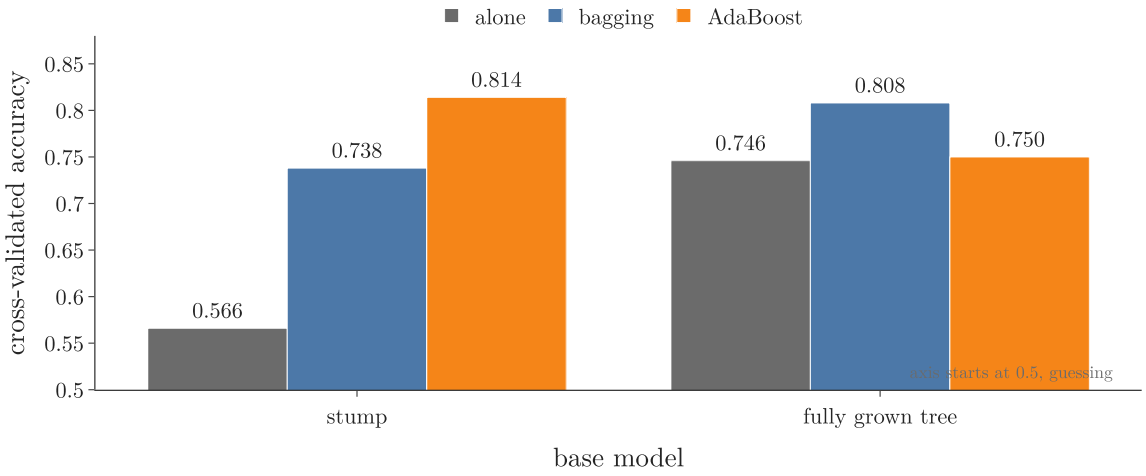


Figure 2: The table as bars. Boosting lifts the stump most (0.566 to 0.814); bagging lifts the fully grown tree most (0.746 to 0.808)

Boosting helps stumps most, bagging helps full trees most. **AdaBoost** (G-167) with full trees is no better than one tree: the first tree already classifies every training observation correctly, its error is 0, and boosting stops after that single tree, since no mistakes are left to pass on.

2.3 Watching the two errors move

Key point: Bagging (G-251) starts with a good training fit and gains on new data as models are added; boosting starts poor on both and climbs on both.

Figure 3 adds base models one at a time and tracks the accuracy on the training data and on test data, averaged over 20 fresh draws of the noisy circles (350 training and 150 test points each).

- **Bagging (left).** One fully grown tree already scores 0.92 on the training data: the bias is low from the start. The score is not 1.00, as for the single tree of the Extra above, because a bagged tree is grown on a random sample drawn with replacement, which leaves out about 36.8% of the training observations (drawing with replacement);

the score counts all of them. The gap down to its test accuracy, 0.78, is the variance. Adding trees averages part of that variance away, and the test accuracy rises to 0.82.

- **Boosting (right).** One stump scores 0.63 on the training data and 0.60 on the test data: both are poor and close together, the mark of high bias and low variance. Each added stump lowers the bias, so both curves climb together, to 0.88 and 0.84 after 100 stumps.

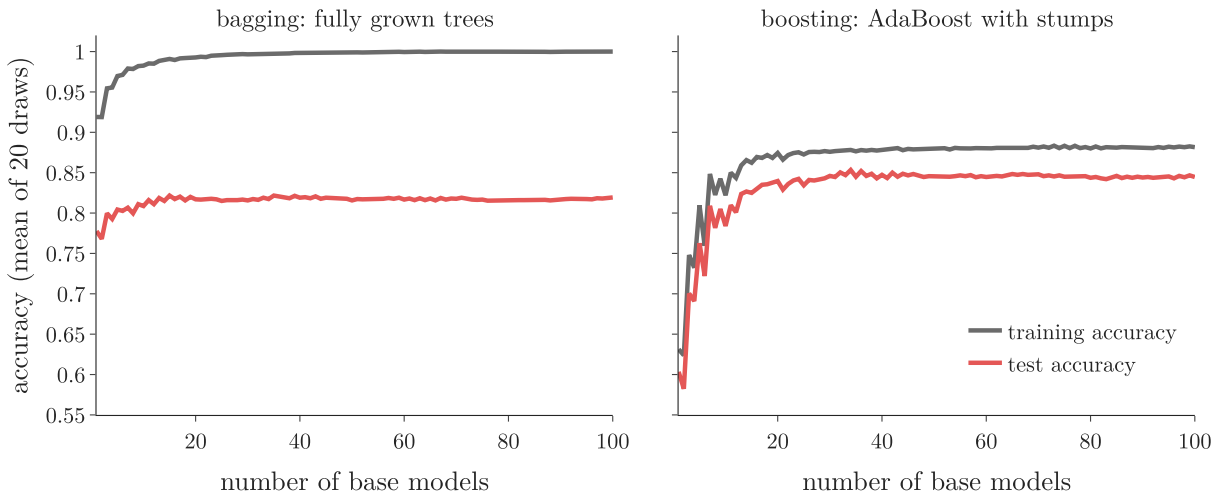


Figure 3: Training accuracy (grey) and test accuracy (red) against the number of base models, averaged over 20 draws of the noisy circles. Left: bagging with fully grown trees. Right: AdaBoost with stumps.

3. Difference 2: parallel against sequential learning

> **Key point: Bagging** (G-251) trains all its models side by side, independently; boosting trains them one after another, each depending on the one before.

Bagging is parallel (**parallel learning**, G-1443; Figure 1, left). Bagging stands for bootstrap aggregation (bootstrapping and aggregation). From a dataset of, say, 1,000 **observations** (G-1374; records, one row of the data table each), each base model gets its own random sample of observations. No model needs anything from any other, so all of them can be trained at the same time.

Boosting is sequential (**sequential learning**, G-1775; Figure 1, right). The data goes to model 1, which does its job and passes its mistakes on to model 2. Model 2 does its job and passes its own mistakes to model 3, and so on. Model 2 cannot start before model 1 has finished, because it learns from model 1's mistakes.

Figure 4 runs both on the noisy circles of the Notebook, one base model per frame. Watch the marker sizes on the left. In bagging (top) each tree's sample is a fresh random draw: big and small markers land anywhere, whatever the earlier trees got wrong. In boosting (bottom) the weights pile up on the observations the stumps so far keep missing, near the border of

the disc, so each stump depends on all the ones before it. The right panels show difference 1 as well: bagging's deep trees fit the training data at once (training accuracy 0.89 with one tree, 1.00 by 50), while boosting's stumps start weak (0.61) and climb stage by stage (0.85 after 100).

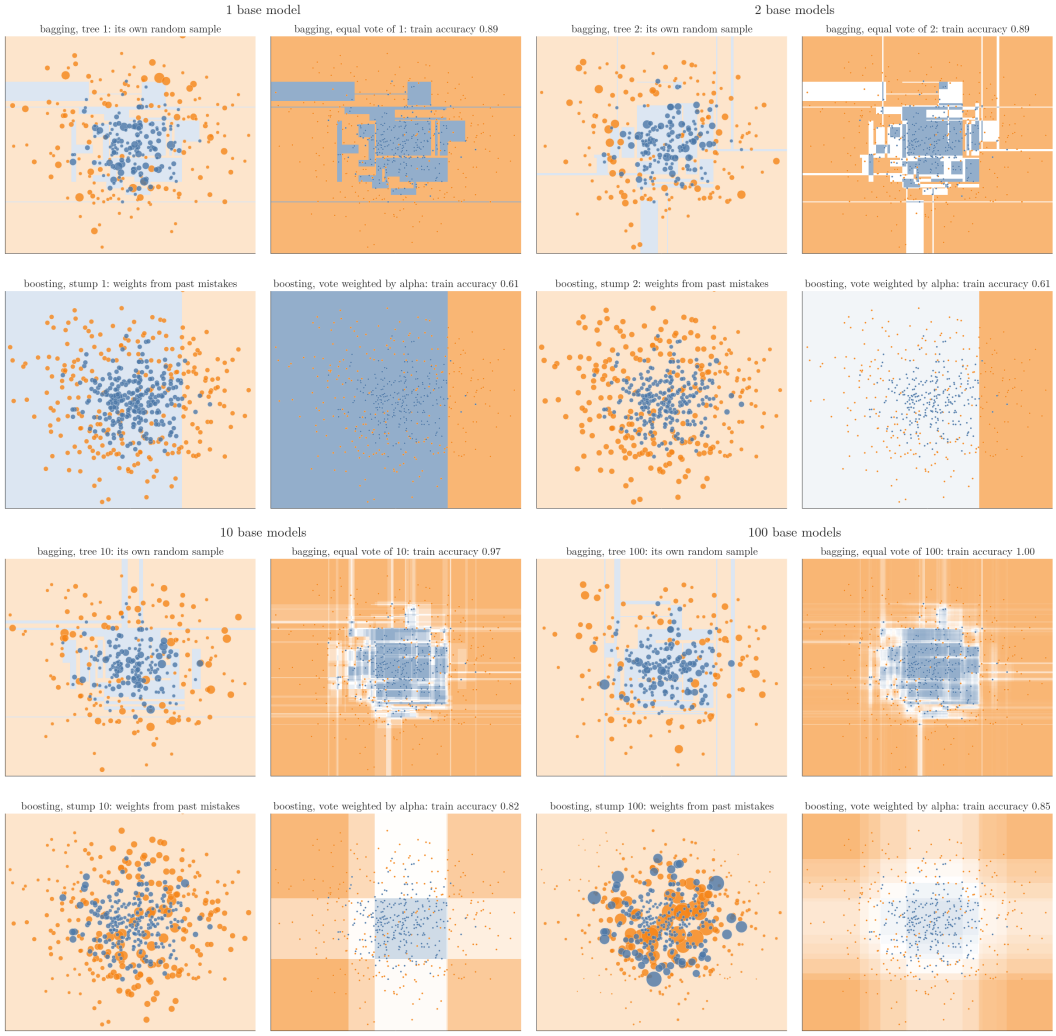


Figure 4: Bagging (top: fully grown trees, each on its own random sample; marker size = copies drawn) against AdaBoost (bottom: stumps; marker size = weight from earlier mistakes) on the noisy circles, one base model per frame. Right: the vote so far; white means a split vote

> **Extra:** Parallel training has a practical side. Bagging can spread its models over every CPU core (`n_jobs=-1` in `BaggingClassifier` and `RandomForestClassifier`); `AdaBoostClassifier` has no `n_jobs`, because its stages must run one at a time.

4. Difference 3: the weight of each base model

> **Key point:** In **bagging** (G-251) every base model's vote counts the same, like a democracy; in boosting each model's vote is weighted by how well it did.

When a new **query point** (G-1605) arrives, every trained base model gives its answer. The two methods combine those answers differently.

Bagging: equal votes. Suppose four models answer 1, 1, 0 and 1. Every vote has the same weight, like a democracy, so the **majority vote** (G-1146) wins: three say 1, the output is 1 (Figure 1, left: every weight is 1).

Boosting: weighted votes. Every model carries its own weight, its **alpha** (G-192; the say of the stump). With weights such as 0.8, 1.5 and 5, the model with weight 5 is listened to far more than the one with 0.8. A model that made fewer mistakes during training earns a larger say. Suppose those three models answer 1, 1 and 0: class 1 collects the weights of the two models that said 1:

$$0.8 + 1.5 = 2.3$$

Class 0 collects 5, so the output is 0, although two of the three models said 1 (Figure 5).

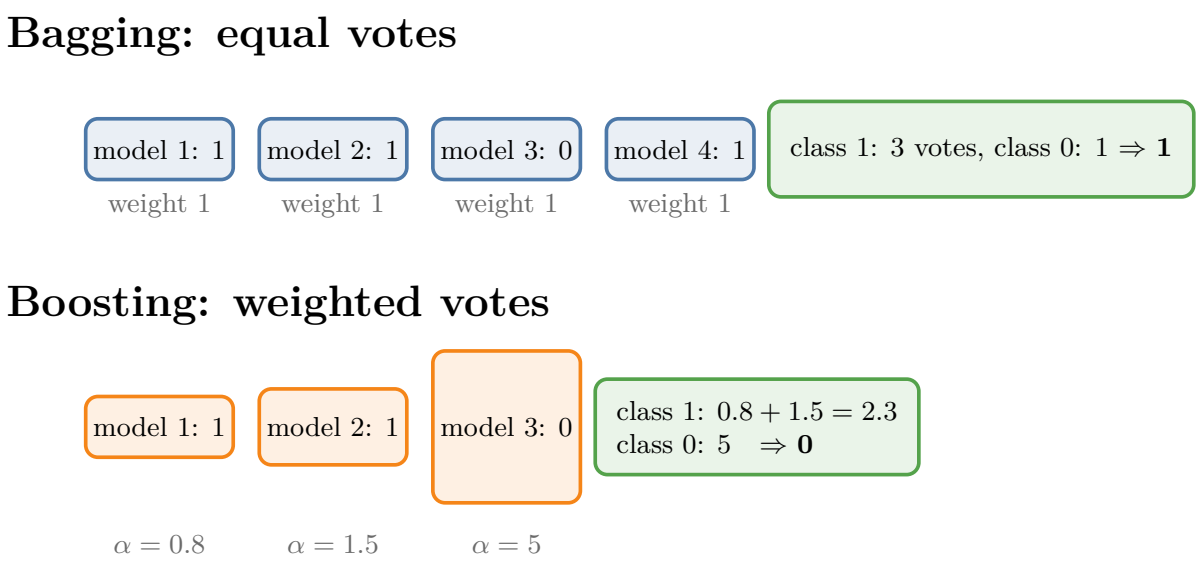


Figure 5: Equal against weighted votes. Bagging: four models answer 1, 1, 0, 1 with weight 1 each, so 1 wins three votes to one. Boosting: three models answer 1, 1, 0 with alphas 0.8, 1.5 and 5 (box height), so 0 wins, 5 to 2.3

5. Summary

	Bagging	Boosting	Why it matters
Base models	low bias, high variance (fully grown trees, KNN with small k)	high bias, low variance (stumps, shallow trees)	the most important difference: it tells you which method suits a model
Mainly reduces	variance	bias	each method removes the weakness its base models start with
Learning	parallel: models trained independently, at the same time	sequential: each model learns from the previous one's mistakes	bagging can use every CPU core (`n_jobs=-1`); boosting must run its stages one at a time
Data each model sees	a random sample of the observations	all observations, reweighted toward earlier mistakes	boosting's later models focus on the points the earlier ones keep missing
Weight in the vote	equal (democracy)	each model's alpha, earned by its accuracy	in boosting a model that made fewer mistakes is listened to more
Examples	bagging, random forest	AdaBoost, gradient boosting, XGBoost	

- Both aim at low bias and low variance, from opposite ends, because a single model usually trades one for the other.
- **Bagging** (G-251) averages unstable, accurate models; boosting adds up stable, weak ones, so a model that is good on the training data but unstable calls for bagging, and a stable but weak one calls for boosting.
- Bagging trains in parallel and votes equally; boosting trains in sequence and weights each vote, because each boosting model needs the previous one's mistakes before it can start.
- These three points (base model, way of learning, weight of the vote) are the answer to the interview question "What is the difference between bagging and boosting?".

6. Sources

Built from

- CampusX, "Bagging Vs Boosting | What is the difference between Bagging and Boosting", YouTube, https://www.youtube.com/watch?v=7M5oWXCpDEw

Other references

- Grinsztajn, L., Oyallon, E. and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *NeurIPS 2022, Datasets and Benchmarks Track*. arxiv.org/abs/2207.08815
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning* (ESL), 2nd ed. Springer. §15.2 (bagging reduces variance only; boosting removes bias).

7. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Shallow decision tree (G-1785)	A decision tree with a small maximum depth: high bias, low variance; boosting uses such trees as its base models.
Parallel learning (G-1443)	Training the base models independently, so they can all be trained at once (bagging).
Sequential learning (G-1775)	Training the base models one after another, each learning from the mistakes of the ones before; this is how boosting works.
Bagging (bootstrap aggregation) (G-251)	Training many models on different random samples of the data and averaging them, so the result depends less on the particular sample (lower variance).
Random forest (G-1611)	Bagging with decision trees as the base models: many trees, each trained on a random sample of the rows with a random choice of features at every split, vote (classification) or are averaged (regression), which lowers the variance of a single tree.
Gradient boosting (G-859)	A way to build a model from many trees added one by one: it starts from a simple guess, and each new tree is trained on the mistakes (pseudo-residuals) of the ensemble so far (a boosting algorithm).
XGBoost (G-2133)	eXtreme Gradient Boosting: a library that implements gradient boosting with many speed and accuracy optimisations.
AdaBoost (Adaptive Boosting) (G-167)	A boosting algorithm that trains simple models (weak learners) one after another on reweighted data, and combines them by a vote in which each model counts by its alpha weight.

Term	Meaning
Low bias, high variance algorithm (G-1132)	An algorithm that fits its training data very well but changes a lot with the data, such as a fully grown tree; it overfits.
Decision stump (G-559)	A decision tree with maximum depth 1: one question, two regions. Too weak alone, it is AdaBoost's usual weak learner.
Cross-validation (G-510)	Testing a model by training and testing it several times on different parts of the training data.
Observation (G-1374)	One record of a dataset: one row of the data table.
Query point (G-1605)	The new point whose class we want to predict.
Majority vote (G-1146)	Predicting the class that most voters choose: the k neighbours in KNN, or the base models of an ensemble.
Alpha (model weight) (G-192)	A base model's say in AdaBoost's final vote; larger when it made fewer mistakes.